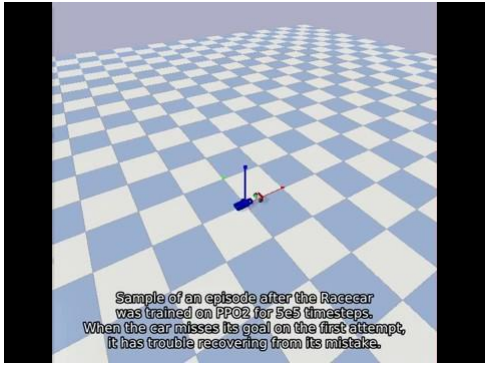
Real-Time Operating Systems for Robotics

Roman Aguilera

Motivation / Problem



- Agile Robotics
  - Goal: Train RC to able to perform reliable skidding maneuvers for an RC car
  - This requires 2 critical tasks
    - Observing the state (position and orientation of car)
      - Querying the immediate position and orientation of the car at consistent and fine granularity of time
    - Sending a control signal to the car
      - Wheel throttle and wheel turn

RGBD Camera

Solution: Use Real-Time Operating Systems

- Real-Time Operating Systems Deliver Reliability
  - Determinism
    - A given task's computation will be done in the same manner every time
    - A task will be implemented exactly at a specified window of time
  - Deadlines
    - A critical task will be completed before a specified time
- Real-Time Operating Systems were designed for two general classes of applications:
  - Event response
    - Example: Stock Market trading companies (Real -time trading decisions)
  - Closed-loop control
    - Example: Cruise control (Necessary throttle must be accurately assessed)

Background: Real-Time Operating Systems

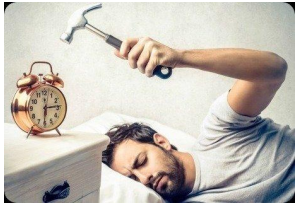
	General Purpose Operating Systems	Real-Time Operating Systems
High Level Definition	Responsible for managing hardware resources and software applications that run on a computer.	OS capabilities + Run applications with high degree of reliability.
Goal	Ensure fairness among all tasks.	Ensure prioritized tasks are done reliably.

Background: Definitions of Real-Time

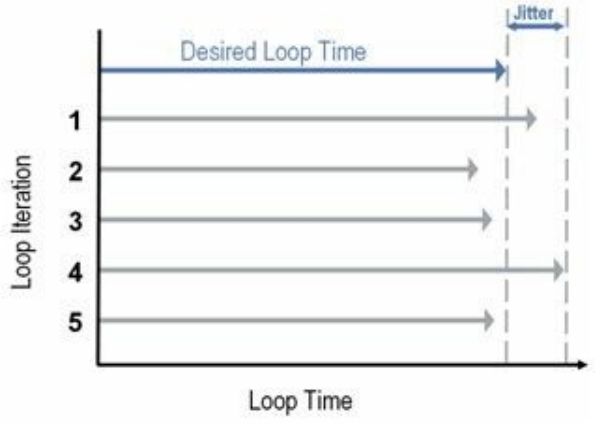
	Soft Real-Time	Firm Real-Time	Hard Real-Time
Definition	The usefulness of a result degrades after its deadline, thereby degrading the system's quality of service. Missing a deadline degrades the quality of service.	Infrequent deadline misses are tolerable, but may degrade the system's quality of service. Missing a deadline is very detrimental to quality of service.	The usefulness of a response to a critical-task request is zero after its deadline. Missing a deadline is a failure.
Examples	Common sound systems	Seismic sensors	Automotive applications, Nuclear reactors, Teleoperated Medical Robots

Background: Latency and Jitter

Latency



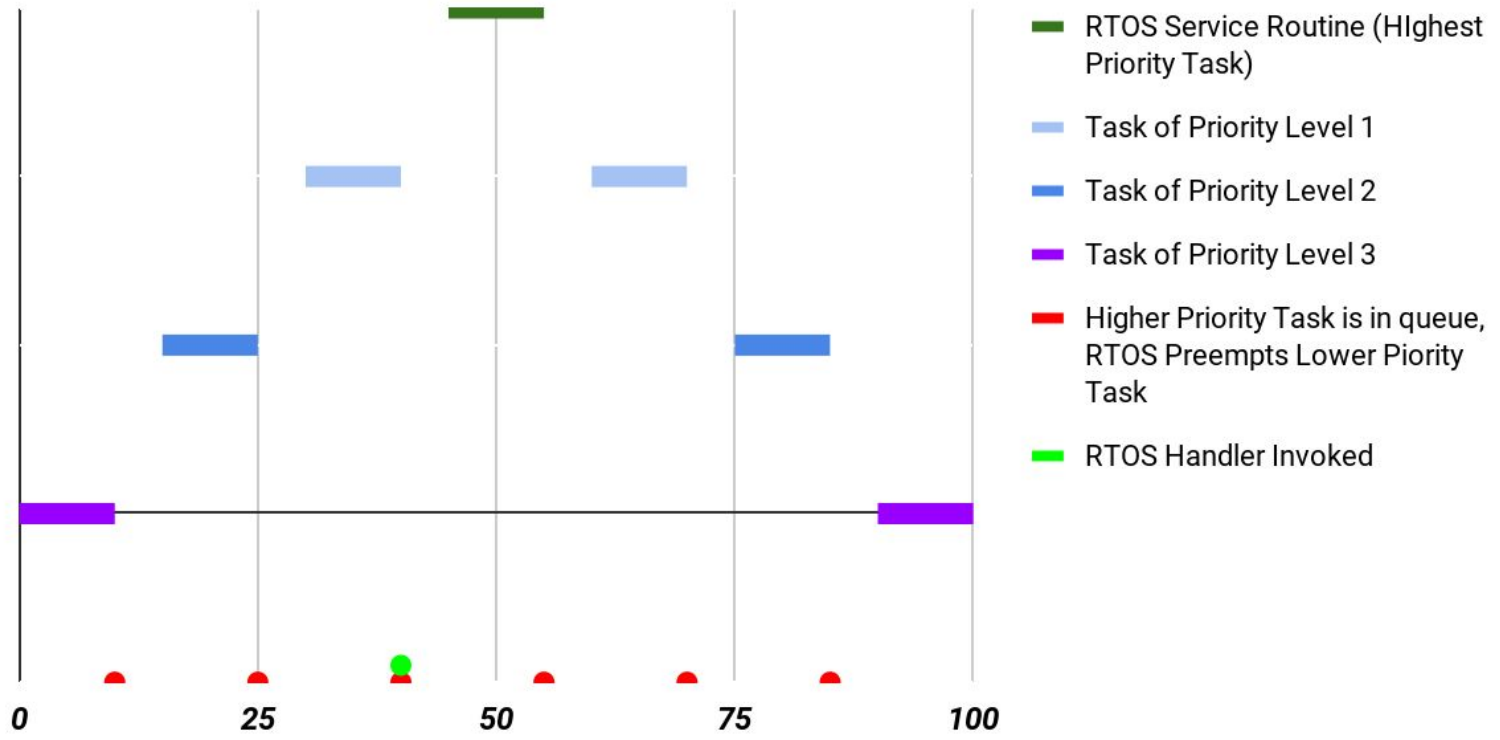
- Latency = Time between event and systems response to that event
  - For Real-Time operating Systems:
    - Consistency and predictability of interrupt latency is of utmost importance
    - Lower interrupt latency is preferred but not as important as consistency
- Jitter = Time deviation from desired signal time to actual
  - For Real-Time Operating Systems
    - The jitter is variation in latency
    - RTOS provides maximum bound on jitter



Background: Real-Time Operating Systems Programming Paradigm

- RTOS makes timing guarantees by giving programmers 2 tools:
  - High degree of control over how tasks are prioritized
  - Ability to check that critical-task program deadlines are met
    - Critical tasks
      - OS calls
      - Interrupt handling
      - Others

Illustration of Task Switching in Real-Time Operating Systems



Background: Approaches for Creating Real-Time Operating Systems

	Single Kernel Approach	Dual Kernel Approach
Architecture Approach	Uses single kernel that is preemptive.	Uses microkernel and general OS kernel. Microkernel is the one that handles the real time aspects.
Problems	No open source platform for hard real-time.	Special API knowledge required. Microkernel needs to be adapted for every combination of linux kernel and target hardware. Doesn’t scale to big platforms.

Survey: Open Source Real-Time Operating Systems Based on Linux

- RT_PREEMPT
  - Linux kernel patch, which modifies the Linux scheduler to be fully preemptible
  - Can’t boast hard real time guarantees but that is being developed
  - https://wiki.linuxfoundation.org/realtime/start
- Xenomai
  - Co-kernel cooperating with linux kernel
  - Linux kernel is treated as an idle task of the real-time kernel’s scheduler
  - POSIX-compliant
  - https://gitlab.denx.de/Xenomai/xenomai/-/wikis/home
- RTAI
  - Another linux based co-kernel solution
  - Better performance than Xenomai, but not as clean and extensible
  - https://www.rtai.org/

Survey: Proprietary Hard Real-Time Operating Systems

- Free licenses for students and professors offered by
  - VxWorks
    - https://www.windriver.com/universities/
    - https://www.windriver.com/products/vxworks/
  - QNX Neutrino
    - https://blackberry.qnx.com/en/company/qnx-in-education
    - https://blackberry.qnx.com/en/software-solutions/embedded-software/industrial/qnx-neutrino-rtos

Implementation: Choosing a Setup

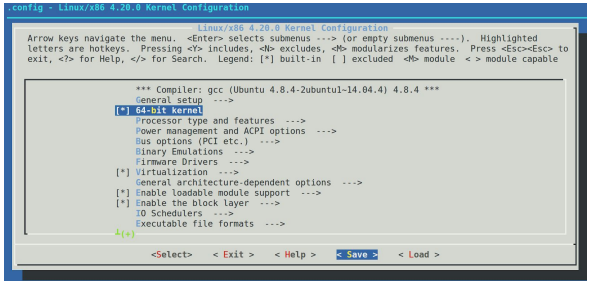
- Vision
  - Intel RealSense
  - IMU inside
  - x_86 based
- Image processing for state observation and Feedback Control
  - Done on Linux x86 based personal computer
- Real-Time Operating System of Choice: Xenomai
  - Open source
  - Hard Real-Time
  - Latency and jitter performance competitive with industrial RTOS's
  - Built on top of linux
  - POSIX compliant
  - Runs on any target x86 platform
  - Clean documentation, extensible

Implementation: Installing Xenomai

- Realsense C++ libraries only compatible with select linux kernels
  - Linux-4.4 was the only one that Xenomai supported
- Xenomai installation is straightforward, but not for x_86
  - Requires extra custom kernel configuration

Findings: Linux Kernel Configuration for Xenomai

- Custom kernel configuration and compilation was not arbitrary
  - Configuration options are different for every kernel
  - Xenomai patches add extra kernel configuration options that are not standard to linux
  - Online guides and tutorials are helter-skelter
  - No two kernels are the same in their configuration options
  - Have to be careful, otherwise might see compilation errors that you can’t backtrack
  - A lot of trial and error
  - Some configuration option dependencies are non-intuitive



Contributions

- Surveyed Real-Time Operating Systems
- Configured and compiled Linux kernel version 4.4 for Xenomai 3, to be deployed on x86_64 architectures
- Created tutorial to save someone hours on installing Xenomai
    - Tutorial includes tips for configuring and compiling custom linux kernels
    - https://github.com/roman-aguilera/RTOS4ROBOTS